

# High-Level Synthesis with Tabu Search Scheduling

Soaib Ferdous

Student ID: 1232368

HW/SW Codesign Seminar Paper

BEng in Electronic Engineering

Hochschule Hamm-Lippstadt (HSHL)

**Abstract**—High-level synthesis transforms a behavioural description into a register-transfer level implementation by assigning operations to functional units and clock cycles. Scheduling is the central optimisation problem because it directly determines resource usage, latency, and hardware cost. This paper studies a tabu search approach for multiprocessor scheduling in HLS, as proposed by Sevaux, Singh, and Rossi. The method formulates HLS scheduling as a minimisation of the number of identical parallel processors subject to release dates, due dates, and precedence constraints. An integer linear programming model provides exact solutions for small instances, while the proposed tabu search procedure TS-PRED handles large instances where ILP becomes impractical. The paper explains the formal problem model, the ILP formulation, all three phases of the TS-PRED algorithm, a compact C++ implementation, and a worked application example. It also evaluates the method against ILP and against the earlier TS-SS heuristic, with attention to scalability, hidden assumptions, and proposed extensions.

**Index Terms**—High-level synthesis, tabu search, multiprocessor scheduling, resource minimisation, metaheuristics, RTL.

## I. INTRODUCTION

Modern embedded systems and electronic chip designs require functional units such as adders and multipliers to be implemented in hardware. Manual register-transfer level design is powerful but labour-intensive; high-level synthesis addresses this gap by transforming an algorithmic or behavioural description into a structural hardware implementation automatically [2].

In the standard HLS flow, a behavioural specification is mapped to an RTL netlist in which operations are bound to functional units and executed according to a controller. The output is a netlist of RTL components, where each component can be realised as a VHDL entity [6]. This process involves three closely related subproblems: scheduling, allocation, and binding. Scheduling assigns operations to time steps; allocation determines how many functional units of each type are needed; binding maps each scheduled operation to a specific unit.

Scheduling is difficult because a good schedule must respect data dependencies, respect time windows imposed by the design, and minimise hardware cost simultaneously. If all operations run in parallel the latency is minimised but the hardware count increases. If operations are serialised the hardware count falls but latency grows. Sevaux, Singh, and Rossi model this as a multiprocessor scheduling problem whose objective is to minimise the number of identical parallel

processors while respecting release dates, due dates, and precedence constraints [1]. This formulation maps directly to HLS: processors correspond to functional units, jobs correspond to operations, and clock cycles correspond to discrete time steps.

The problem is NP-hard, which means exact methods such as integer linear programming become impractical for large instances. The proposed tabu search procedure TS-PRED fills this gap. It is the only known heuristic that handles the version of the problem with precedence constraints and has been demonstrated on both synthetic and real electronic circuit instances.

## II. HLS SCHEDULING PROBLEM

### A. Job and Processor Model

The HLS scheduling problem is modelled as follows. A list of  $n$  jobs (operations) must be scheduled on  $m$  identical parallel processors (functional units). For each job  $i \in [1, n]$  four parameters are given: a release date  $r_i$  (earliest start time), a due date  $d_i$  (latest completion time), a processing time  $p_i$  (duration in clock cycles), and a set of predecessors  $\text{PRED}_i$  (operations that must complete before  $i$  can start).

All times are non-negative integers, which is natural because all events in a digital chip are triggered by a clock signal. The time horizon is computed as

$$H = \max_{j \in [1, n]} (d_j - p_j). \quad (1)$$

Jobs can start only at integer times in  $\{0, 1, \dots, H\}$ . The scheduling variable  $v_{j,t} \in \{0, 1\}$  is set to one if and only if job  $j$  starts at time  $t$ . The start time of job  $j$  is then

$$s_j = \sum_{t \in [r_j, d_j - p_j]} t \cdot v_{j,t}. \quad (2)$$

### B. Precedence Structure

Dependencies between operations are represented as a directed acyclic graph  $G = (V, E)$ , where each node in  $V$  models one job and each directed arc  $(i, j) \in E$  means job  $i$  must complete before job  $j$  can start. The acyclic structure is guaranteed by the data-flow nature of HLS operations and allows a topological ordering of jobs to be computed efficiently.

### C. Objective and Constraints

The objective is to minimise  $m$ , the total number of processors required. Four constraint classes must be satisfied simultaneously. The time-window constraint ensures each job starts exactly once:

$$\sum_{t \in [r_j, d_j - p_j]} v_{j,t} = 1 \quad \forall j \in [1, n]. \quad (3)$$

The capacity constraint limits concurrency to  $m$  processors:

$$\sum_{\substack{j \in [1, n] \\ s \in [t - p_j + 1, t] \cap [r_j, d_j - p_j]}} v_{j,s} \leq m \quad \forall t \in [0, H]. \quad (4)$$

The precedence constraint enforces dependency order:

$$\sum_t (t + p_i) v_{i,t} \leq \sum_t t \cdot v_{j,t} \quad \forall (i, j) \in E. \quad (5)$$

Finally,  $v_{j,t} \in \{0, 1\}$  for all  $j$  and  $t$ .

### D. Complexity

The problem is NP-hard. For the version without precedence constraints, setting all  $r_i = 0$  and all  $d_i$  to a common value reduces the problem to one-dimensional bin packing, which is known to be NP-hard [3]. The version with precedence constraints is also NP-hard because the unconstrained version is a special case with  $\text{PRED}_i = \emptyset$  for all  $i$ . No polynomial-time approximation scheme is known [1]. This motivates the use of a metaheuristic approach.

## III. RESOLUTION APPROACHES

### A. Integer Linear Programming Baseline

An exact ILP model solves the problem for small instances. The model minimises  $m$  subject to constraints (3)–(6) with binary variables  $v_{j,t}$ . It is implemented using the open-source GLPK library and allowed a maximum runtime of 600 seconds. ILP finds optimal solutions for all instances up to  $n = 100$  without precedence constraints, and for many instances up to  $n = 100$  with precedence constraints. For  $n = 1000$  with precedence, ILP fails to find any feasible solution within the time limit, demonstrating that an alternative method is essential for practical HLS.

### B. Tabu Search: TS-PRED

The proposed tabu search procedure TS-PRED (Tabu Search with Precedence) follows a reverse approach compared to earlier methods [4]. Rather than building up from a lower bound on the processor count, it starts from a feasible solution with  $m$  processors and repeatedly tries to find solutions using fewer processors. It terminates when the best solution does not improve for 7500 consecutive iterations or when a total runtime of 600 seconds is reached.

1) *Preprocessing*: Before the search begins, release dates and due dates are tightened using a method analogous to critical path computation. A forward pass computes the earliest start time (EST) of each job: jobs with no predecessors receive  $\text{EST}_i = r_i$ , while other jobs receive  $\text{EST}_i = \max_{k \in \text{PRED}_i} (\text{EST}_k + p_k)$ . If  $\text{EST}_i > r_i$  then  $r_i \leftarrow \text{EST}_i$ . A symmetric backward pass tightens due dates from successors. This preprocessing significantly reduces the search space and identifies infeasible instances early so they can be excluded from testing.

2) *Initial Solution Generation*: Jobs are numbered in topological order so that each job's number is always higher than all its predecessors. An iterative greedy procedure then allocates jobs one by one. At each step, the lowest-numbered job whose predecessors are all already allocated is selected. It is inserted on the processor where the idle time after insertion is minimum. If no existing processor can accommodate the job within its time window, a new processor is added. The start time is always set to the earliest feasible time considering the job's release date, predecessor completions, and the preceding job on the selected processor. Once allocated, a job cannot be moved during this phase.

3) *Tabu Search Procedure*: Once the initial solution is established, the tabu search procedure begins. A perturbation step creates a set  $U$  of unallocated jobs using one of two methods. Method 1 removes one or two processors randomly (two if  $m \geq 7$ , one otherwise), and is always used when  $m \geq 25$  and used with probability 0.95 otherwise. Method 2 removes each job independently with probability  $p_{\text{rem}} = 0.2$ .

Unallocated jobs are then reallocated in random order. For each job  $j \in U$ , insertion is attempted first: the algorithm tries to place  $j$  between allocated jobs on some processor by pushing others earlier or later, using the earliest completion time (ECT) and latest start time (LST) of neighbouring jobs. Among all feasible insertion positions, the one leaving the smallest idle time is chosen. If insertion fails, an exchange is attempted:  $j$  is swapped with consecutively allocated jobs whose schedules overlap with  $j$ 's time interval, but only when each such job has processing time strictly less than  $p_j$ . This restriction reflects the observation that shorter jobs are easier to reschedule. If neither insertion nor exchange is possible, a new processor is added.

After reallocation, recently inserted jobs are made tabu to prevent the search from immediately reversing decisions. The tabu tenure is

$$\tau = \max \left\{ 3, \left\lceil \frac{n}{m} \right\rceil \right\}, \quad (6)$$

where  $n$  is the number of jobs and  $m$  is the current processor count. If the new solution uses no more processors than the current solution it replaces it; otherwise it is discarded. The procedure repeats until the stopping condition is satisfied.

### C. Algorithm Summary

Algorithm 1 summarises the full TS-PRED procedure.

**Algorithm 1** TS-PRED Tabu Search

---

```

1: Preprocess: tighten  $r_i, d_i$  via critical path
2: Generate initial feasible solution  $s$ 
3: repeat
4:   Perturb  $s \rightarrow s'$ , creating unallocated set  $U$ 
5:   Initialise tabu list
6:   while  $U \neq \emptyset$  do
7:     Select  $j$  randomly from  $U$ 
8:     if insertion of  $j$  succeeds then
9:       Make  $j$  tabu for tenure  $\tau$ 
10:    else if exchange of  $j$  succeeds then
11:      Make  $j$  tabu; add exchanged jobs to  $U$ 
12:    else
13:      Add new processor; allocate  $j$  on it
14:    end if
15:  end while
16:  if  $s'$  uses  $\leq m$  processors then
17:     $s \leftarrow s'$ 
18:  end if
19: until stopping condition satisfied

```

---

## IV. C++ IMPLEMENTATION

To satisfy the implementation requirement, a standalone C++14 program was written that implements all three phases of TS-PRED. The program is not a complete commercial HLS tool; it implements the main scheduling feature including preprocessing, greedy initial solution generation, and the tabu search loop with insertion, exchange, and tabu tenure management.

The core data structures are a `Job` struct holding  $(r_i, d_i, p_i, \text{PRED}_i)$ , a `Processor` struct holding an ordered list of scheduled jobs, and a `Solution` holding all processors. Listing 1 shows the tabu tenure computation and the structure of the main search loop.

Listing 1. Tabu tenure and main TS-PRED loop

```

1 // Tabu tenure: max{3, ceil(n/m)}
2 int tabu_tenure(int n, int m) {
3     return std::max(3, (n + m - 1) / m);
4 }
5
6 // Main loop (simplified)
7 while (no_improve < 7500 && iter < max_iter) {
8     Solution perturbed = current;
9     vector<int> U = perturb_method1(perturbed);
10    TabuList tabu(n);
11    int tau = tabu_tenure(n, current.m());
12    for (int job_id : U) {
13        if (!try_insert(jobs[job_id], perturbed, ...))
14            if (!try_exchange(jobs[job_id], perturbed, U, ...))
15                add_new_processor(jobs[job_id], perturbed);
16        tabu.make_tabu(job_id, tau);
17    }
18    if (perturbed.m() <= current.m())
19        current = perturbed;
20 }

```

The preprocessing phase implements forward and backward passes over the job list in topological order. The initial solution generator inserts each job at the position minimising idle time across all existing processors. The tabu list tracks the

remaining tenure of each job and is decremented after each reallocation step.

## V. RUNTIME AND OVERHEAD ANALYSIS

The runtime of TS-PRED is determined by three phases. The preprocessing phase performs a forward and backward pass over the  $n$  jobs in topological order, giving  $O(n + |E|)$  time where  $|E|$  is the number of precedence arcs. The initial solution generation considers each job against all existing processors; in the worst case this is  $O(n \cdot m)$  where  $m$  grows with  $n$ , but in practice the greedy insertion terminates quickly because idle-time minimisation selects the first suitable position.

The dominant cost is the tabu search loop. Each iteration perturbs the current solution and reallocates all unallocated jobs. For each unallocated job, insertion checks every position on every processor in  $O(m \cdot H)$  time; exchange checks are of similar order. With up to 7500 non-improving iterations and a 600-second hard limit, the total work is bounded by the stopping condition rather than a closed-form expression.

In practice, Sevaux et al. report runtimes under two seconds for  $n = 100$  without precedence and under 105 seconds for  $n = 1000$  with precedence [1]. This is several times faster than ILP at the same sizes, and the speed advantage grows with  $n$ . Memory usage is proportional to  $n \cdot H$  for the time-interval tracking structures, which remains moderate for all HLS instance sizes tested in the paper.

## VI. APPLICATION EXAMPLE

To demonstrate the scheduler, consider a small HLS data-flow graph with four operations: two multiplications and two additions, with the following dependency structure:  $J_0 \rightarrow J_2$ ,  $J_1 \rightarrow J_2$ , and  $J_2 \rightarrow J_3$ . The job parameters are listed in Table I.

TABLE I  
APPLICATION EXAMPLE JOB PARAMETERS

| Job   | Operation | $r_i$ | $d_i$ | $p_i$ |
|-------|-----------|-------|-------|-------|
| $J_0$ | Multiply  | 0     | 4     | 2     |
| $J_1$ | Multiply  | 0     | 4     | 2     |
| $J_2$ | Add       | 0     | 6     | 2     |
| $J_3$ | Add       | 2     | 8     | 2     |

The precedence constraints mean  $J_2$  cannot start until both  $J_0$  and  $J_1$  are complete, and  $J_3$  cannot start until  $J_2$  is complete. The C++ implementation was executed with these parameters. The output is shown in Fig. 1.

The scheduler correctly determines that one functional unit is sufficient.  $J_0$  runs in clock cycles 0–2,  $J_1$  in cycles 2–4,  $J_2$  in cycles 4–6, and  $J_3$  in cycles 6–8. All precedence constraints are respected:  $J_2$  starts only after both  $J_0$  and  $J_1$  complete at cycle 4, and  $J_3$  starts only after  $J_2$  completes at cycle 6. All time-window constraints are satisfied. This result is optimal for this instance: the sequential precedence chain  $J_0 \rightarrow J_2 \rightarrow J_3$  forces at least three serial operations, and  $J_1$  fits into the idle time before  $J_2$ , so no parallelism is needed.

```

=== TS-PRED Application Example ===
4 HLS operations, precedence: J0->J2, J1->J2, J2->J3

Initial solution: 1 processors
TS-PRED finished after 7500 iterations.
Best solution: 1 processors

=== Schedule (1 processors) ===
Processor 0: [J0 s=0 f=2] [J1 s=2 f=4] [J2 s=4 f=6] [J3 s=6 f=8]

...Program finished with exit code 0
Press ENTER to exit console.

```

Fig. 1. C++ TS-PRED output for the 4-operation HLS example.

In a larger HLS instance with more independent operations and tighter deadlines, multiple processors would be needed and TS-PRED would search for the minimum count through perturbation and reallocation.

## VII. EXPERIMENTAL RESULTS AND COMPARISON

### A. Results Without Precedence Constraints

Sevaux et al. compare TS-PRED against TS-SS [4] and ILP on synthetic instances of sizes  $n \in \{20, 40, 60, 80, 100\}$ . ILP finds the optimal solution for all instances at this scale. TS-PRED achieves near-optimal results with a maximum deviation of 1.24% from the ILP optimal at  $n = 100$ , while TS-SS deviates by 5.45% at the same size. TS-PRED is also considerably faster than TS-SS, averaging 1.09 seconds at  $n = 100$  compared to 49.9 seconds for TS-SS [1].

### B. Results With Precedence Constraints

Four instance types (Types 1–4) reflect DSP application characteristics with varying graph densities and processing time ranges. For small instances ( $n \leq 100$ ), TS-PRED matches or closely approaches ILP quality. Degradation begins at  $n = 250$ , where the number of optimal solutions found falls, particularly for Type 2 instances. For  $n = 1000$ , ILP cannot find any feasible solution for most instance types within the 600-second limit. TS-PRED, by contrast, finds a feasible solution for all 20 instances of every type at  $n = 1000$ , demonstrating its practical value for large-scale HLS problems.

### C. Real-Life Instances

TS-PRED was also tested on 30 real electronic circuit instances provided by circuit designers, classified as filters, matrices, and miscellaneous applications. TS-PRED returns the optimal solution for all 27 instances for which ILP could also find an optimal, while executing significantly faster [1].

### D. Key Differences: TS-PRED vs TS-SS

Table II summarises the main algorithmic differences between the two methods.

TABLE II  
TS-PRED vs TS-SS ALGORITHMIC COMPARISON

| Aspect             | TS-SS                   | TS-PRED              |
|--------------------|-------------------------|----------------------|
| Precedence support | None                    | Full                 |
| Search direction   | Build from lower bound  | Reduce from feasible |
| Tabu criterion     | Sum of processing times | Move-based           |
| Exchange condition | Unrestricted            | Shorter jobs only    |
| Preprocessing      | None                    | Critical path        |

## VIII. CRITICAL EVALUATION AND CONCLUSION

### A. Strengths

TS-PRED fills an important gap: it is the only heuristic available for the multiprocessor scheduling problem with precedence constraints relevant to HLS. It always produces a feasible solution within the time budget, achieves near-optimal quality on small and medium instances, and scales to sizes where ILP completely fails. The preprocessing phase and the move-based tabu criterion both contribute to faster convergence compared to earlier methods.

### B. Limitations

The method carries several important limitations. It provides no optimality guarantee and quality degrades noticeably at  $n \geq 250$  with precedence. All parameters — tabu tenure, perturbation probabilities, and stopping thresholds — were chosen empirically with no theoretical justification, and sensitivity analysis is not reported. The model assumes identical parallel processors, integer time steps, non-preemptive jobs, and a fixed single objective (minimise processor count). Real HLS problems frequently involve heterogeneous functional units, power and area objectives, and register-pressure constraints, none of which are addressed.

### C. Proposed Extensions

The authors propose two extensions: a memetic algorithm with population management (MA|PM) using TS-PRED as local search [5], and support for multiple processor types. Additional natural directions include multi-objective optimisation incorporating latency and power alongside processor count, adaptive parameter control replacing the fixed empirical values, and tighter lower bound computations to improve solution quality evaluation on large instances.

### D. Conclusion

Tabu search for multiprocessor scheduling in HLS is best understood as a practical metaheuristic compromise. It is more principled than greedy list scheduling because it can escape local optima through perturbation and tabu-guided reallocation. It is more scalable than exact ILP because it does not enumerate all possible start-time assignments. For HLS of resource-constrained datapaths where precedence constraints must be respected and instance sizes are large, TS-PRED represents a scientifically sound and practically useful scheduling approach.

## AI USAGE AND PROTOCOL

This paper was prepared with AI assistance for topic structuring, algorithm explanation, wording, and implementation support. Technical claims were verified against the assigned reference Sevaux et al. (2011) before inclusion. The detailed AI-supported workflow is documented separately in the submitted AI usage protocol JSON and BibTeX files [7].

## REFERENCES

- [1] M. Sevaux, A. Singh, and A. Rossi, “Tabu Search for Multiprocessor Scheduling: Application to High Level Synthesis,” *Asia-Pacific Journal of Operational Research*, vol. 28, no. 2, pp. 201–212, 2011, doi: 10.1142/S0217595911003132.
- [2] D. D. Gajski, N. D. Dutt, A. C.-H. Wu, and S. Y.-L. Lin, *High-Level Synthesis: Introduction to Chip and System Design*. Norwell, MA, USA: Kluwer Academic Publishers, 1992.
- [3] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco: Freeman, 1979.
- [4] M. Sevaux and K. Sørensen, “A Tabu Search Method for High Level Synthesis,” in *Conf. Conjointe Francoro V – Roadev 2007*, pp. 395–396, Grenoble, France, 2007.
- [5] K. Sørensen and M. Sevaux, “MA|PM: Memetic Algorithms with Population Management,” *Computers and Operations Research*, vol. 33, no. 5, pp. 1214–1225, 2006.
- [6] A. Rettberg, *HW/SW Codesign Lecture Slides*. Hochschule Hamm-Lippstadt, June 2026.
- [7] S. Ferdous, “AI Usage Protocol for High-Level Synthesis with Tabu Search Scheduling,” Submitted AI usage protocol notebook, JSON and BibTeX files, HW/SW Codesign Seminar, HSHL, 2026.